

Implementasi dan Analisis Arsitektur Penggunaan Register serta Memori pada Algoritma Bubble Sort Mikrokontroler 8051 Menggunakan MCU 8051 IDE

Laila Maulidia Aghnia Putri
Departemen Ilmu Komputer dan
Elektronika

Universitas Gadjah Mada
Yogyakarta, Indonesia

lailamaulidiaaghniaputri@mail.ugm.ac.id

Marsela Nur Choirunnisa
Departemen Ilmu Komputer dan
Elektronika

Universitas Gadjah Mada
Yogyakarta, Indonesia

marselanurchoirunnisa@mail.ugm.ac.id

Wahyu Yulianto
Departemen Ilmu Komputer dan
Elektronika

Universitas Gadjah Mada
Yogyakarta, Indonesia

wahyuyulianto@mail.ugm.ac.id

Abstract—Penelitian ini menganalisis performa dua implementasi algoritma Bubble Sort pada mikrokontroler 8051 menggunakan simulator MCU 8051 IDE, yaitu Bubble Sort standar (V1) dan Bubble Sort dengan optimasi *early exit* (V2). Pengujian dilakukan pada array berisi lima elemen 8-bit dengan tiga skenario data, yaitu *best case*, *average case*, dan *worst case*. Parameter yang diamati meliputi waktu eksekusi, kondisi register, dan kebenaran hasil pengurutan. Hasil menunjukkan bahwa kedua implementasi mampu mengurutkan data dengan benar pada seluruh skenario pengujian. Mekanisme *early exit* pada V2 memungkinkan penghentian proses lebih awal ketika tidak ditemukan pertukaran elemen dalam satu iterasi, sehingga dapat mengurangi jumlah pass yang diperlukan. Namun, penambahan instruksi untuk pemeriksaan flag menimbulkan overhead yang menyebabkan V2 tidak selalu lebih cepat dibandingkan V1. Meskipun demikian, pada kondisi tertentu optimasi tersebut mampu memberikan peningkatan efisiensi yang signifikan. Hasil penelitian menunjukkan bahwa penerapan *early exit* dapat menjadi solusi efektif untuk meningkatkan performa proses pengurutan pada sistem embedded berbasis 8051, terutama ketika data yang diproses telah mendekati kondisi terurut.

Index Terms—Bubble Sort, Mikrokontroler 8051, Early Exit, Assembly Language, Analisis Performa, Embedded System

I. PENDAHULUAN

A. Latar Belakang

Arsitektur 8051 merupakan salah satu arsitektur mikrokontroler yang banyak digunakan dalam pembelajaran sistem komputer dan sistem tertanam karena memiliki struktur yang sederhana, set instruksi yang lengkap, serta mudah dipelajari. Berbagai platform berbasis 8051, termasuk Oregano 8051 dan simulator mikrokontroler 8051, menyediakan lingkungan yang memungkinkan pengguna untuk mempelajari eksekusi program pada level instruksi dan register secara lebih mendalam.

Dalam sistem berbasis 8051, efisiensi algoritma menjadi aspek penting karena keterbatasan sumber daya perangkat keras yang tersedia. Salah satu operasi yang umum dijumpai adalah pengurutan data (*sorting*), baik untuk pengolahan data sensor, penyusunan data memori, maupun aplikasi sederhana lainnya.

Oleh karena itu, analisis terhadap implementasi algoritma pengurutan pada arsitektur 8051 menjadi menarik untuk dilakukan.

Pada penelitian ini digunakan algoritma Bubble Sort yang diimplementasikan menggunakan bahasa Assembly 8051 dan dijalankan pada simulator MCU 8051 IDE yang kompatibel dengan arsitektur 8051. Dua versi implementasi dibandingkan, yaitu Bubble Sort standar dan Bubble Sort dengan optimasi *early exit*. Perbandingan dilakukan untuk menganalisis pengaruh optimasi terhadap performa eksekusi serta penggunaan sumber daya pada arsitektur 8051.

B. Tujuan

Tujuan dari penelitian ini adalah:

- 1) Mengimplementasikan algoritma Bubble Sort pada arsitektur 8051 menggunakan bahasa Assembly.
- 2) Menganalisis proses eksekusi program pada platform simulasi berbasis 8051.
- 3) Membandingkan performa Bubble Sort standar dan Bubble Sort dengan optimasi *early exit*.
- 4) Mengevaluasi efektivitas optimasi *early exit* dalam meningkatkan efisiensi eksekusi program.

C. Ruang Lingkup

Penelitian dilakukan menggunakan simulator MCU 8051 IDE versi 1.4.9 dengan mikrokontroler AT89S52 pada frekuensi clock 12 MHz. Data yang digunakan berupa array lima elemen 8-bit yang disimpan pada memori internal alamat 20H–24H. Pengujian dilakukan pada tiga skenario, yaitu *best case*, *average case*, dan *worst case*. Parameter yang diamati meliputi waktu eksekusi, kondisi register, nilai *Program Counter*, serta kebenaran hasil pengurutan data. Pengujian dilakukan menggunakan MCU 8051 IDE sebagai simulator yang mengimplementasikan arsitektur dan set instruksi 8051 yang kompatibel dengan platform Oregano 8051, sehingga program Assembly yang dikembangkan dapat dijalankan dan dianalisis dengan karakteristik yang setara pada tingkat arsitektur.

II. LANDASAN TEORI

A. Arsitektur Mikrokontroler 8051

Arsitektur 8051 merupakan mikrokontroler 8-bit yang diperkenalkan oleh Intel dan hingga saat ini masih banyak digunakan dalam pembelajaran sistem tertanam serta pengembangan aplikasi embedded sederhana [1]. Arsitektur ini menyediakan CPU 8-bit, memori program, memori data, port I/O, timer, dan antarmuka serial dalam satu sistem terintegrasi. Salah satu karakteristik utamanya adalah pemisahan memori program dan memori data yang memungkinkan akses instruksi dan data dilakukan secara lebih efisien [1].

Pada penelitian ini digunakan beberapa register utama untuk mendukung proses pengurutan data, yaitu ACC, B, R0, R1, R2, dan R3. Register-register tersebut berperan dalam proses perbandingan data, pengalamatan memori, pengendalian loop, serta implementasi mekanisme *early exit*.

TABLE I: Register 8051 yang Digunakan

Register	Fungsi	Penggunaan
ACC (A)	Accumulator	Perbandingan dan swap
B	Register bantu	Data sementara
R0	Pointer	Akses array
R1	Counter	Outer loop
R2	Counter	Inner loop
R3	Flag	Status swap
CY	Carry Flag	Hasil perbandingan

B. Algoritma Bubble Sort

Bubble Sort merupakan algoritma pengurutan berbasis perbandingan yang bekerja dengan membandingkan pasangan elemen yang berdekatan dan menukarnya apabila urutannya tidak sesuai [2]. Proses ini dilakukan secara berulang hingga seluruh elemen berada pada posisi yang benar.

Algoritma Bubble Sort memiliki kompleksitas waktu rata-rata dan terburuk sebesar $O(n^2)$, sedangkan kompleksitas ruangnya sebesar $O(1)$ karena proses pengurutan dilakukan secara *in-place* [2]. Meskipun kurang efisien untuk data berukuran besar, algoritma ini banyak digunakan dalam proses pembelajaran karena implementasinya sederhana dan mudah direalisasikan menggunakan bahasa Assembly.

C. Optimasi Early Exit

Optimasi *early exit* merupakan teknik yang digunakan untuk menghentikan proses Bubble Sort lebih awal ketika seluruh elemen telah berada dalam urutan yang benar [3]. Teknik ini memanfaatkan sebuah indikator atau *flag* yang menunjukkan apakah terjadi pertukaran data selama satu pass. Apabila tidak terdapat pertukaran elemen dalam satu iterasi penuh, maka array dianggap telah terurut dan proses pengurutan dapat dihentikan. Dengan pendekatan ini, kompleksitas kasus terbaik dapat ditingkatkan dari $O(n^2)$ menjadi $O(n)$ [3]. Pada penelitian ini register R3 digunakan sebagai *flag* untuk mencatat apakah operasi pertukaran data terjadi selama proses pengurutan.

D. Instruksi Assembly 8051

Implementasi Bubble Sort pada penelitian ini memanfaatkan beberapa instruksi dasar Assembly 8051 yang digunakan untuk manipulasi data, percabangan, dan pengendalian loop [1]. Instruksi MOV, SUBB, dan XCH digunakan untuk proses perbandingan dan pertukaran data, sedangkan DJNZ, JNC, dan JZ digunakan untuk mengendalikan alur program.

TABLE II: Instruksi Assembly 8051 yang Digunakan

Instruksi	Format	Fungsi	Siklus
MOV	MOV dst, src	Memindahkan data	1-2
SUBB	SUBB A, src	Perbandingan data	1
XCH	XCH A, src	Pertukaran data	1
DJNZ	DJNZ Rn, label	Loop counter	2
JNC	JNC label	Lompat jika CY=0	2
JZ	JZ label	Lompat jika A=0	2
CLR C	CLR C	Reset Carry Flag	1
INC	INC Rn	Increment register	1
DEC	DEC Rn	Decrement register	1
LJMP	LJMP label	Long jump	2
SJMP	SJMP label	Short jump	2

III. IMPLEMENTASI

A. Lingkungan Pengembangan

Eksperimen dilakukan menggunakan MCU 8051 IDE versi 1.4.9 yang menyediakan editor Assembly, assembler, debugger, dan simulator mikrokontroler 8051 dalam satu lingkungan pengembangan. Simulator digunakan untuk mengamati perubahan register, isi memori, dan waktu eksekusi program selama proses pengujian. Platform yang digunakan adalah AT89S52 dengan frekuensi clock 12 MHz dan memori data internal 128 byte. Seluruh program ditulis menggunakan bahasa Assembly 8051 tanpa menggunakan pustaka eksternal.

B. Implementasi Bubble Sort Standar (V1)

Implementasi pertama menggunakan algoritma Bubble Sort standar yang menjalankan seluruh iterasi *outer loop* tanpa mempertimbangkan kondisi data. Array berukuran lima elemen disimpan pada alamat memori internal 20H–24H. Register R1 digunakan sebagai penghitung *outer loop*, R2 sebagai penghitung *inner loop*, dan R0 sebagai pointer untuk mengakses elemen array secara tidak langsung. Proses perbandingan dilakukan menggunakan instruksi SUBB, sedangkan pertukaran data dilakukan menggunakan instruksi XCH. Implementasi ini selalu menjalankan empat pass penuh hingga nilai R1 mencapai nol.

1	[ORG	0000H
2		LJMP MAIN
3		ORG 0030H
4	MAIN:	
5		MOV 20H, #05H
6		MOV 21H, #03H
7		MOV 22H, #08H
8		MOV 23H, #01H
9		MOV 24H, #07H
10		MOV R1, #04H
11	OUTER:	
12		MOV A, R1
13		MOV R2, A
14		MOV R0, #20H
15	INNER:	
16		MOV A, @R0

```

17      INC      R0
18      MOV      B, @R0
19      CLR      C
20      SUBB     A, B
21      JNC      NOSWAP
22      DEC      R0
23      MOV      A, @R0
24      XCH      A, B
25      MOV      @R0, A
26      INC      R0
27      MOV      @R0, B
28  NOSWAP:
29      DJNZ     R2, INNER
30      DJNZ     R1, OUTER
31  DONE:
32      SJMP     DONE
33      END]

```

Listing 1: Implementasi Bubble Sort Standar (V1)

Program diawali dengan direktif `ORG 0000H` yang menentukan alamat awal eksekusi program pada memori program mikrokontroler 8051. Instruksi `LJMP MAIN` digunakan untuk mengalihkan alur eksekusi menuju label `MAIN` yang ditempatkan pada alamat `0030H`. Pendekatan ini umum digunakan pada pemrograman 8051 untuk memisahkan area reset vector dengan program utama sehingga memudahkan pengembangan sistem yang lebih kompleks. Setelah mencapai label `MAIN`, program melakukan inisialisasi data array ke dalam memori internal pada alamat `20H` hingga `24H`. Nilai yang disimpan adalah `{05, 03, 08, 01, 07}`, yang akan digunakan sebagai data masukan untuk proses pengurutan.

Setelah proses inisialisasi selesai, register `R1` diisi dengan nilai `04H` yang merepresentasikan jumlah maksimum pass yang diperlukan oleh algoritma Bubble Sort untuk mengurutkan array yang terdiri dari lima elemen. Nilai ini diperoleh dari rumus $N-1$, di mana N adalah jumlah elemen array. Register `R1` kemudian berfungsi sebagai pencacah (*counter*) untuk *outer loop*. Pada awal setiap iterasi *outer loop*, isi register `R1` disalin ke accumulator (`ACC`) dan kemudian dipindahkan ke register `R2`. Register `R2` digunakan sebagai pencacah *inner loop* yang mengendalikan jumlah perbandingan elemen pada setiap pass. Selanjutnya register `R0` diisi dengan alamat `20H` sehingga dapat digunakan sebagai pointer yang menunjuk elemen pertama array.

Proses utama pengurutan dilakukan pada bagian `INNER`. Instruksi `MOV A, @R0` digunakan untuk mengambil elemen pertama yang sedang dibandingkan dan menyimpannya ke accumulator. Pointer `R0` kemudian dinaikkan satu langkah menggunakan instruksi `INC R0` sehingga menunjuk ke elemen berikutnya. Nilai elemen kedua kemudian dimuat ke register `B` melalui instruksi `MOV B, @R0`. Sebelum proses perbandingan dilakukan, *Carry Flag* dibersihkan menggunakan instruksi `CLR C` untuk memastikan hasil operasi tidak dipengaruhi oleh kondisi sebelumnya. Perbandingan dilakukan menggunakan instruksi `SUBB A, B`, yang mengurangi nilai dalam accumulator dengan nilai pada register `B`. Hasil operasi ini tidak digunakan secara langsung, melainkan kondisi *Carry Flag* yang dihasilkan digunakan untuk menentukan apakah dua elemen perlu dipertukarkan atau tidak.

Keputusan untuk melakukan pertukaran ditentukan oleh instruksi `JNC NOSWAP`. Jika *Carry Flag* bernilai nol, program

akan melompat ke label `NOSWAP` dan tidak melakukan pertukaran. Sebaliknya, jika kondisi pertukaran terpenuhi, program mengeksekusi serangkaian instruksi untuk menukar posisi kedua elemen. Pointer `R0` terlebih dahulu dikembalikan ke posisi elemen pertama menggunakan instruksi `DEC R0`. Nilai elemen pertama kemudian dimuat kembali ke accumulator dan ditukar dengan isi register `B` menggunakan instruksi `XCH A, B`. Setelah proses pertukaran selesai, nilai yang telah ditukar disimpan kembali ke memori menggunakan instruksi `MOV @R0, A` dan `MOV @R0, B`. Mekanisme ini memastikan bahwa elemen yang lebih kecil dan lebih besar ditempatkan pada posisi yang sesuai sesuai dengan logika pengurutan yang diterapkan.

Setelah satu perbandingan selesai dilakukan, instruksi `DJNZ R2, INNER` digunakan untuk mengurangi nilai pencacah `R2`. Jika nilai `R2` belum mencapai nol, program akan kembali ke label `INNER` untuk membandingkan pasangan elemen berikutnya. Dengan cara ini seluruh elemen yang diperlukan pada satu pass akan diperiksa secara berurutan. Setelah seluruh perbandingan dalam satu pass selesai dilakukan, instruksi `DJNZ R1, OUTER` akan mengurangi nilai pencacah `R1`. Selama nilai `R1` belum nol, program akan kembali ke awal *outer loop* untuk menjalankan pass berikutnya.

Ketika seluruh pass telah selesai dieksekusi dan nilai `R1` mencapai nol, proses pengurutan dianggap selesai. Program kemudian menuju label `DONE`. Pada bagian ini digunakan instruksi `SJMP DONE` yang membentuk loop tak berhingga (*infinite loop*). Tujuan penggunaan instruksi tersebut adalah menghentikan alur program pada kondisi akhir sehingga mikrokontroler tidak mengeksekusi area memori lain yang tidak berisi instruksi program. Setelah seluruh proses selesai, isi memori pada alamat `20H` hingga `24H` berubah menjadi urutan data yang telah terurut, yaitu `{01, 03, 05, 07, 08}`. Hasil ini menunjukkan bahwa implementasi Bubble Sort standar berhasil melakukan pengurutan data secara benar menggunakan instruksi-instruksi dasar Assembly 8051.

C. Implementasi Bubble Sort dengan Early Exit (V2)

Implementasi kedua menambahkan mekanisme *early exit* untuk mengurangi jumlah iterasi yang tidak diperlukan. Register `R3` digunakan sebagai *flag* untuk mendeteksi apakah terjadi pertukaran data selama satu pass. Nilai `R3` diinisialisasi menjadi `00H` pada awal setiap *outer loop* dan diubah menjadi `01H` ketika terjadi operasi pertukaran data. Setelah *inner loop* selesai, nilai `R3` diperiksa. Jika tidak terdapat pertukaran data, program langsung menghentikan proses pengurutan karena array telah berada dalam kondisi terurut.

```

1  [ORG      0000H
2      LJMP   MAIN
3      ORG      0030H
4  MAIN:
5      MOV     20H, #05H
6      MOV     21H, #03H
7      MOV     22H, #08H
8      MOV     23H, #01H
9      MOV     24H, #07H
10     MOV     R1, #04H
11  OUTER:
12     MOV     R3, #00H
13     MOV     A, R1
14     MOV     R2, A

```

```

15      MOV     R0, #20H
16  INNER:
17      MOV     A, @R0
18      INC     R0
19      MOV     B, @R0
20      CLR     C
21      SUBB    A, B
22      JNC     NOSWAP
23      DEC     R0
24      MOV     A, @R0
25      XCH     A, B
26      MOV     @R0, A
27      INC     R0
28      MOV     @R0, B
29      MOV     R3, #01H
30  NOSWAP:
31      DJNZ    R2, INNER
32      MOV     A, R3
33      JZ      DONE
34      DJNZ    R1, OUTER
35  DONE:
36      SJMP    DONE
37  END]

```

Listing 2: Implementasi Bubble Sort dengan Early Exit (V2)

Struktur dasar program V2 identik dengan implementasi V1, yaitu diawali dengan inisialisasi array pada alamat memori internal 20H hingga 24H, kemudian menjalankan algoritma Bubble Sort menggunakan mekanisme *outer loop* dan *inner loop*. Register R1 tetap digunakan sebagai pencacah (*counter*) *outer loop*, register R2 digunakan sebagai pencacah *inner loop*, register R0 berfungsi sebagai pointer menuju elemen array, sedangkan register A dan B digunakan dalam proses perbandingan dan pertukaran data. Perbedaan utama antara V1 dan V2 terletak pada penambahan mekanisme *early exit* yang memungkinkan program menghentikan proses pengurutan lebih awal apabila array telah berada dalam kondisi terurut.

Pada awal setiap iterasi *outer loop*, register R3 diinisialisasi dengan nilai 00H melalui instruksi `MOV R3, #00H`. Register R3 berfungsi sebagai *flag swap*, yaitu penanda apakah selama satu pass terjadi proses pertukaran elemen atau tidak. Nilai 00H menunjukkan bahwa belum ada pertukaran yang terjadi, sedangkan nilai 01H menunjukkan bahwa setidaknya satu pertukaran telah dilakukan. Setelah *flag* direset, program melanjutkan proses yang sama seperti pada V1, yaitu menyalin nilai R1 ke R2 dan mengatur R0 sebagai pointer ke awal array.

Proses perbandingan dua elemen yang bersebelahan dilakukan menggunakan instruksi `SUBB A, B` setelah kedua elemen dimuat ke register A dan B. Jika kondisi mengharuskan pertukaran data, program menjalankan rangkaian instruksi `DEC R0`, `XCH A, B`, dan penyimpanan kembali ke memori seperti pada V1. Namun, setelah proses pertukaran selesai dilakukan, program mengeksekusi instruksi tambahan `MOV R3, #01H`. Instruksi ini merupakan inti dari mekanisme *early exit* karena menandai bahwa pada pass tersebut masih ditemukan pasangan elemen yang tidak berada pada urutan yang benar. Dengan demikian, program mengetahui bahwa proses pengurutan masih harus dilanjutkan pada pass berikutnya.

Setelah seluruh perbandingan dalam satu pass selesai dilakukan, program tidak langsung melanjutkan ke *outer loop* berikutnya. Sebagai gantinya, isi register R3 dipindahkan ke accumulator menggunakan instruksi `MOV A, R3`. Nilai tersebut kemudian diperiksa melalui instruksi `JZ DONE`. Jika nilai R3

masih 00H, berarti selama satu pass penuh tidak terjadi satu pun pertukaran elemen. Kondisi ini menunjukkan bahwa seluruh elemen array telah berada pada posisi yang benar sehingga proses pengurutan dapat dihentikan lebih awal. Dalam keadaan tersebut, instruksi `JZ DONE` akan langsung mengalihkan alur program menuju label `DONE` tanpa menunggu seluruh pass selesai dijalankan.

Sebaliknya, apabila nilai R3 bernilai 01H, instruksi `JZ DONE` tidak akan dieksekusi dan program melanjutkan ke instruksi `DJNZ R1, OUTER`. Instruksi ini mengurangi nilai pencacah R1 dan mengulang *outer loop* sebagaimana pada implementasi V1. Dengan mekanisme ini, V2 hanya menjalankan pass tambahan apabila memang masih terdapat elemen yang perlu ditukar. Pendekatan tersebut dapat mengurangi jumlah iterasi secara signifikan ketika data masukan telah terurut atau hampir terurut.

Pada kondisi *best case*, yaitu ketika data sudah berada dalam urutan yang benar sejak awal, tidak ada satu pun operasi pertukaran yang terjadi selama pass pertama. Akibatnya, nilai R3 tetap 00H dan program langsung keluar melalui instruksi `JZ DONE`. Sebaliknya, pada *average case* dan *worst case*, nilai R3 akan berubah menjadi 01H karena masih ditemukan elemen yang harus ditukar, sehingga seluruh pass tetap dijalankan sampai proses pengurutan selesai. Karakteristik inilah yang membedakan V2 dari V1 dan menjadi dasar optimasi *early exit* yang diterapkan pada algoritma Bubble Sort.

Seperti pada V1, program diakhiri dengan label `DONE` yang berisi instruksi `SJMP DONE`. Instruksi tersebut membentuk *infinite loop* sehingga eksekusi berhenti pada kondisi akhir program. Setelah proses selesai, isi memori pada alamat 20H hingga 24H berisi data yang telah terurut secara ascending. Implementasi V2 menunjukkan bagaimana penambahan beberapa instruksi sederhana dan satu register tambahan dapat digunakan untuk mengimplementasikan optimasi algoritmik yang mampu mengurangi jumlah iterasi pada kondisi tertentu tanpa mengubah hasil pengurutan yang diperoleh.

D. Variasi Data Pengujian

Pada seluruh pengujian, struktur program V1 dan V2 tidak mengalami perubahan. Variasi *best case*, *average case*, dan *worst case* diperoleh dengan mengubah nilai inisialisasi array pada alamat memori 20H–24H sebagaimana ditunjukkan pada Listing 3, Listing 4, dan Listing 5.

```

1  MOV 20H, #01H
2  MOV 21H, #03H
3  MOV 22H, #05H
4  MOV 23H, #07H
5  MOV 24H, #08H

```

Listing 3: Inisialisasi Data Best Case

```

1  MOV 20H, #05H
2  MOV 21H, #03H
3  MOV 22H, #08H
4  MOV 23H, #01H
5  MOV 24H, #07H

```

Listing 4: Inisialisasi Data Average Case

```

1 MOV 20H, #08H
2 MOV 21H, #07H
3 MOV 22H, #05H
4 MOV 23H, #03H
5 MOV 24H, #01H

```

Listing 5: Inisialisasi Data Worst Case

E. Perbandingan Implementasi

Perbedaan utama antara V1 dan V2 terletak pada penggunaan mekanisme *early exit*. Implementasi V2 menambahkan sebuah register untuk menyimpan status pertukaran data dan tiga instruksi tambahan untuk mengelola serta memeriksa *flag*. Penambahan ini memungkinkan proses pengurutan dihentikan lebih awal ketika data telah terurut, namun juga menimbulkan overhead instruksi yang dapat memengaruhi waktu eksekusi pada kondisi tertentu.

TABLE III: Karakteristik Implementasi V1 dan V2

Karakteristik	V1	V2
Metode sorting	Bubble Sort	Bubble Sort + Early Exit
Flag swap	Tidak ada	Register R3
Jumlah register aktif	5	6
Instruksi tambahan	0	3 per pass
Penghentian dini	Tidak	Ya
Best case	Selalu 4 pass	Dapat berhenti lebih awal
Average case	4 pass	Bergantung flag swap
Worst case	4 pass	4 pass
Efisiensi data hampir terurut	Rendah	Tinggi

IV. HASIL DAN ANALISIS

A. Dokumentasi Hasil Simulasi

Dokumentasi visual digunakan untuk memverifikasi perubahan nilai memori dan register setelah proses pengurutan selesai. Gambar ?? hingga Gambar ?? menunjukkan hasil simulasi untuk seluruh skenario pengujian pada implementasi V1 dan V2.

B. Dokumentasi Hasil Simulasi

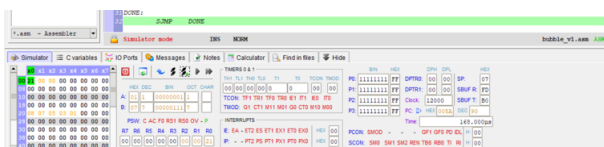


Fig. 1: Hasil simulasi V1 pada skenario average case.

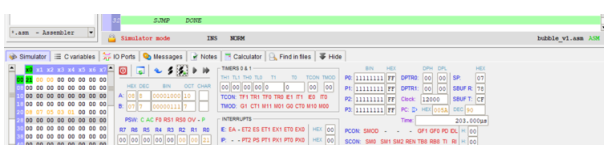


Fig. 2: Hasil simulasi V1 pada skenario best case.

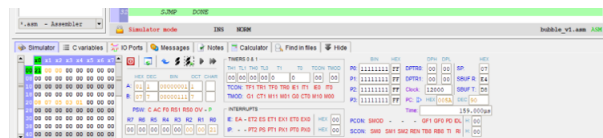


Fig. 3: Hasil simulasi V1 pada skenario worst case.

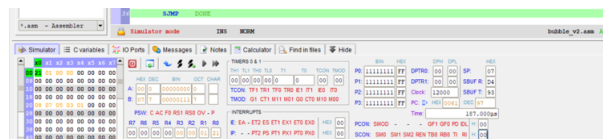


Fig. 4: Hasil simulasi V2 pada skenario average case.

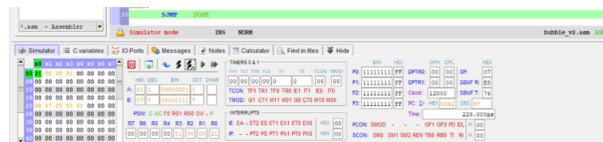


Fig. 5: Hasil simulasi V2 pada skenario best case.

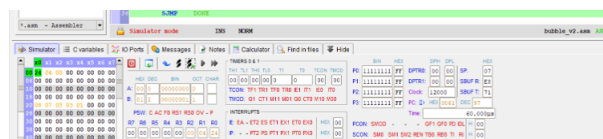


Fig. 6: Hasil simulasi V2 pada skenario worst case.

C. Hasil Eksperimen

Pengujian dilakukan pada tiga skenario data, yaitu *best case*, *average case*, dan *worst case*. Seluruh pengujian dijalankan menggunakan kondisi awal simulator yang sama untuk memastikan konsistensi hasil.

TABLE IV: Hasil Pengujian Bubble Sort Standar (V1)

Parameter	Best Case	Average Case	Worst Case
Input (20H–24H)	01,03,05,07,08	05,03,08,01,07	08,07,05,03,01
Output (20H–24H)	01,03,05,07,08	01,03,05,07,08	01,03,05,07,08
R0 akhir	24H	24H	24H
R1 akhir	00H	00H	00H
R2 akhir	00H	00H	00H
ACC akhir	08H	01H	01H
B akhir	07H	07H	07H
PC akhir	005AH	005AH	005AH
Waktu eksekusi	203 us	168 us	159 us
Jumlah pass	4	4	4

TABLE V: Hasil Pengujian Bubble Sort dengan *Early Exit* (V2)

Parameter	Best Case	Average Case	Worst Case
Input (20H–24H)	01,03,05,07,08	05,03,08,01,07	08,07,05,03,01
Output (20H–24H)	01,03,05,07,08	01,03,05,07,08	01,03,05,07,08
R0 akhir	24H	24H	24H
R1 akhir	03H	00H	00H
R2 akhir	00H	00H	00H
R3 akhir	00H	00H	00H
ACC (A) akhir	01H	00H	00H
B akhir	07H	07H	01H
PC akhir	0061H	0061H	0061H
Waktu eksekusi	229 us	187 us	60 us
Jumlah pass	1 (early exit)	4	4

TABLE VI: Perbandingan Waktu Eksekusi V1 dan V2

Skenario	V1 (us)	V2 (us)	Selisih (us)
Best Case	203	229	+26
Average Case	168	187	+19
Worst Case	159	60	-99

D. Analisis Hasil

Kebenaran Hasil Pengurutan

Seluruh enam skenario pengujian — tiga skenario pada V1 dan tiga skenario pada V2 — menghasilkan keluaran yang identik dan benar, yaitu array terurut secara ascending 01, 03, 05, 07, 08 pada alamat memori internal 20H–24H. Kebenaran ini berlaku tanpa terkecuali pada seluruh variasi input, yang membuktikan bahwa kedua implementasi mengeksekusi logika Bubble Sort secara deterministik dan bebas dari kesalahan logika maupun kesalahan pengalaman memori.

Analisis Nilai Register Akhir

Nilai register akhir memberikan bukti empiris tentang jalur eksekusi yang ditempuh masing-masing program.

Pada V1, nilai R1 akhir = 00H pada ketiga skenario mengkonfirmasi bahwa outer loop selalu dieksekusi penuh sebanyak empat pass tanpa pengecualian. Register R3 tidak digunakan pada V1, sehingga nilainya tidak relevan. Nilai ACC akhir bervariasi antar skenario (08H pada *best case*, 01H pada *average case* dan *worst case*) karena bergantung pada nilai elemen terakhir yang diproses dalam operasi perbandingan. Nilai B akhir = 07H pada *best case* dan *average case*, sedangkan pada *worst case* nilai B akhir = 07H juga, mencerminkan nilai elemen yang terakhir kali dimuat ke register B dalam inner loop. Nilai PC akhir yang seragam yaitu 005AH pada ketiga skenario V1 mengkonfirmasi bahwa program selalu berakhir pada label DONE yang sama.

Pada V2, perbedaan paling signifikan terlihat pada nilai R1 akhir untuk skenario *best case*, yaitu 03H, berbeda dengan nilai 00H pada *average case* dan *worst case*. Nilai R1 = 03H secara langsung membuktikan bahwa mekanisme *early exit* aktif: outer loop baru mengurangi R1 satu kali (dari 04H ke 03H), yang berarti program keluar setelah hanya satu pass dari empat pass yang tersedia. Nilai R3 akhir = 00H pada ketiga skenario V2 konsisten dengan perilaku yang diharapkan: pada *best case* R3 tetap 00H karena tidak ada swap yang terjadi, sementara pada

average case dan *worst case* R3 sempat bernilai 01H selama eksekusi namun direset ke 00H di awal pass terakhir yang tidak menghasilkan swap. Nilai PC akhir V2 = 0061H lebih tinggi dari V1 (005AH) karena kode V2 memiliki instruksi tambahan sehingga label DONE berada di alamat yang lebih tinggi.

Analisis Waktu Eksekusi

Best Case. Pada skenario *best case* dengan input yang telah terurut 01, 03, 05, 07, 08, V1 mencatat waktu eksekusi 203 μ s sedangkan V2 mencatat 229 μ s, sehingga V2 lebih lambat 26 μ s atau sekitar 12,8

Average Case. Pada skenario *average case* dengan input acak 05, 03, 08, 01, 07, V1 mencatat waktu eksekusi 168 μ s dan V2 mencatat 187 μ s, dengan selisih 19 μ s atau 11,3

Worst Case. Pada skenario *worst case* dengan input terbalik 08, 07, 05, 03, 01, diperoleh hasil yang paling menarik: V2 mencatat waktu eksekusi 60 μ s, jauh lebih cepat dibandingkan V1 yang membutuhkan 159 μ s, dengan penghematan 99 μ s atau 62,3

Analisis Penggunaan Register

V1 menggunakan lima register aktif (R0, R1, R2, ACC, B), sementara V2 menggunakan enam register aktif dengan penambahan R3 sebagai flag swap. Pada arsitektur 8051 yang menyediakan delapan register per bank (R0–R7), penambahan satu register pada V2 tidak menimbulkan konflik dalam eksperimen ini. Namun, dalam konteks sistem tertanam yang lebih kompleks dengan banyak subprogram aktif secara bersamaan, konsumsi register yang lebih tinggi perlu dipertimbangkan. Sebagai alternatif, flag swap dapat disimpan pada *bit-addressable memory* (misalnya bit 20H.0) sehingga register R3 dapat dibebaskan untuk keperluan lain tanpa mengorbankan fungsionalitas *early exit*.

E. Implikasi Desain

Hasil pengujian menunjukkan bahwa optimasi *early exit* memberikan manfaat ketika data telah terurut atau mendekati terurut karena mampu mengurangi jumlah iterasi yang dijalankan. Namun, optimasi ini juga menambah overhead instruksi dan penggunaan satu register tambahan. Oleh karena itu, penggunaan *early exit* lebih sesuai untuk aplikasi yang sering memproses data dengan tingkat keterurutan tinggi, sedangkan implementasi standar dapat menjadi pilihan yang lebih sederhana untuk data berukuran kecil.

V. KESIMPULAN DAN SARAN

A. Kesimpulan

Berdasarkan hasil implementasi dan pengujian algoritma Bubble Sort pada mikrokontroler 8051 menggunakan simulator MCU 8051 IDE, dapat disimpulkan bahwa kedua implementasi, yaitu Bubble Sort standar (V1) dan Bubble Sort dengan mekanisme *early exit* (V2), berhasil mengurutkan data dengan benar pada seluruh skenario pengujian. Mekanisme *early exit* yang diimplementasikan menggunakan register R3 sebagai *flag* mampu mendeteksi kondisi array yang telah terurut

dan menghentikan proses pengurutan lebih awal. Namun, hasil pengujian menunjukkan bahwa penambahan mekanisme optimasi tidak selalu menghasilkan waktu eksekusi yang lebih baik karena adanya overhead instruksi tambahan. Pada kondisi tertentu, optimasi *early exit* memberikan keuntungan dengan mengurangi jumlah iterasi yang diperlukan, sedangkan pada kondisi lain implementasi standar tetap menunjukkan performa yang kompetitif. Selain itu, penggunaan satu register tambahan pada V2 merupakan *trade-off* yang masih dapat diterima untuk aplikasi sederhana berbasis 8051. Secara keseluruhan, hasil eksperimen menunjukkan bahwa pemilihan implementasi algoritma harus mempertimbangkan karakteristik data masukan serta keterbatasan sumber daya pada sistem tertanam.

B. Saran

Pengembangan lebih lanjut dapat dilakukan dengan menggunakan ukuran array yang lebih besar untuk memperoleh gambaran performa yang lebih representatif. Selain itu, analisis dapat diperluas melalui perhitungan siklus instruksi secara manual untuk memverifikasi hasil pengukuran simulator. Penelitian berikutnya juga dapat membandingkan Bubble Sort dengan algoritma pengurutan lain, seperti Insertion Sort dan Selection Sort, pada platform 8051 yang sama sehingga diperoleh pemahaman yang lebih komprehensif mengenai efisiensi algoritma pada sistem tertanam.

VI. REPOSITORY PROYEK

Seluruh kode sumber, dokumentasi, hasil pengujian, serta berkas pendukung yang digunakan dalam proyek ini tersedia secara publik pada repositori GitHub berikut:

<https://github.com/rainlilyno/bubble-sort-8051>

REFERENCES

- [1] Intel Corporation, *MCS-51 Microcontroller Family User Manual*. Santa Clara, CA, USA: Intel Corporation, 1980.
- [2] D. E. Knuth, *The Art of Computer Programming, Vol. 3: Sorting and Searching*, 2nd ed. Reading, MA, USA: Addison-Wesley, 1998.
- [3] X. Li and Y. Wang, "Analysis on Bubble Sort Algorithm Optimization," *International Conference on Computer Science and Information Processing*, pp. 453–456, 2012.
- [4] M. K. Hanzal, "MCU 8051 IDE: Integrated Development Environment for 8051 Microcontrollers," SourceForge Project Documentation, 2011. Available: <https://sourceforge.net/projects/mcu8051ide/>